

KDE Connect Manager - Project Structure

Complete File Structure

```

kdeconnect-manager/
├── backend/
│   ├── __init__.py
│   └── api/
│       ├── __init__.py
│       └── server.py
│   ├── kdeconnect/
│       ├── __init__.py
│       └── dbus_interface.py
│   └── monitoring/
│       ├── __init__.py
│       └── health_monitor.py
│   ├── ssh/
│       ├── __init__.py
│       └── remote_manager.py
│   └── screenshare/
│       ├── __init__.py
│       └── screen_manager.py
├── cli/
│   ├── __init__.py
│   └── kdecn.py
├── frontend/
│   ├── static/
│   │   ├── css/
│   │   │   └── style.css
│   │   └── js/
│   │       └── app.js
│   └── templates/
│       └── index.html
├── scripts/
│   ├── install.sh
│   └── uninstall.sh
├── systemd/
│   ├── kdeconnect.service
│   └── kdeconnect-manager.service
├── config/
│   └── config.example.yaml
├── examples/
│   └── basic_usage.py
├── logs/
├── tests/
├── README.md
├── QUICK_START.md
├── PROJECT_STRUCTURE.md
├── LICENSE
├── .gitignore
├── requirements.txt
└── setup.py

```

Backend services and logic
 # Package initialization
 # REST & WebSocket API
 # Flask server with SocketIO
 # KDE Connect integration
 # DBus wrapper for KDE Connect
 # Health monitoring system
 # Continuous monitoring & self-healing
 # Remote management
 # SSH-based remote management
 # Screen sharing integration
 # krfb, x11vnc, PipeWire integration
 # Command-line interface
 # Click-based CLI tool
 # Web interface
 # Static assets
 # Complete styling
 # WebSocket client & UI logic
 # HTML templates
 # Main web interface
 # Installation & utility scripts
 # Main installation script
 # Uninstallation script
 # Systemd service files
 # KDE Connect daemon service
 # Manager web server service
 # Configuration
 # Example configuration file
 # Example scripts
 # Usage examples
 # Application logs (created at runtime)
 # Test directory (for future tests)
 # Comprehensive documentation
 # Quick start guide
 # This file
 # MIT License
 # Git ignore rules
 # Python dependencies
 # Package setup script

File Descriptions

Backend Components

`backend/api/server.py`

- Flask application with SocketIO support
- REST API endpoints for device management
- WebSocket server for real-time updates
- Health monitoring integration
- Event broadcasting to web clients

Key Features:

- Device listing and management
- Pairing/unpairing devices
- File and text sharing
- Battery status monitoring
- Health checks and daemon restart
- Real-time event streaming

`backend/kdeconnect/dbus_interface.py`

- Python wrapper around KDE Connect's DBus API
- Device discovery and information retrieval
- Device pairing and unpairing
- Plugin interaction (battery, notifications, sharing, etc.)
- Signal handling for device events

Key Classes:

- `KDEConnectDBusInterface` : Main interface to KDE Connect daemon

`backend/monitoring/health_monitor.py`

- Continuous health monitoring system
- Self-healing capabilities
- Resource usage tracking
- Firewall status checking
- Network connectivity validation
- Automatic daemon restart

Key Classes:

- `HealthMonitor` : Main health monitoring and self-healing class

Features:

- Configurable check intervals
- Event callbacks for notifications
- History tracking
- Automatic issue resolution

`backend/ssh/remote_manager.py`

- SSH-based remote machine management
- Install KDE Connect on remote systems
- Configure firewall rules remotely
- Manage daemon on remote machines

- Fleet management for multiple hosts

Key Classes:

- RemoteKDEConnectManager : Single host management
- RemoteFleet : Multi-host management

backend/screenshare/screen_manager.py

- Screen sharing integration
- Support for multiple methods (krfb, x11vnc, PipeWire)
- Auto-detection of display server (X11/Wayland)
- Configuration and status management

Key Classes:

- ScreenShareManager : krfb VNC management
- X11VNCManager : x11vnc management
- PipeWireScreenShare : PipeWire/Wayland support
- ScreenShareIntegration : Unified interface

Frontend Components

frontend/templates/index.html

- Modern, responsive web interface
- Multiple tabs: Devices, Health, Logs, Settings
- Device cards with status indicators
- Modal dialogs for device details
- Real-time updates via WebSocket

frontend/static/css/style.css

- Dark theme design
- Responsive layout
- Custom components styling
- Animation and transitions
- Mobile-friendly design

frontend/static/js/app.js

- WebSocket client implementation
- Tab management
- Device display and interaction
- Health status visualization
- Log management
- Settings persistence
- Real-time notifications

CLI Component

cli/kdecn.py

- Click-based command-line interface
- Hierarchical command structure
- Color-coded output
- Table formatting for device lists
- Interactive confirmations

- JSON output option

Command Groups:

- `device` : Device management commands
- `share` : File and text sharing
- `health` : Health monitoring and diagnostics
- `server` : Web server management
- `install` : Installation and setup

Scripts

`scripts/install.sh`

- Automated installation for Fedora 42+
- System dependency installation
- Python package installation
- Firewall configuration
- Systemd service setup
- CLI tool installation
- Installation verification

`scripts/uninstall.sh`

- Clean uninstallation
- Service stopping and removal
- Optional package removal
- Firewall rule cleanup
- CLI tool removal

Configuration

`systemd/kdeconnect.service`

- User systemd service for KDE Connect daemon
- Auto-restart on failure
- Resource limits
- Security settings

`systemd/kdeconnect-manager.service`

- User systemd service for web server
- Depends on KDE Connect daemon
- Auto-restart on failure
- Environment configuration

`config/config.example.yaml`

- Complete configuration template
- Server settings
- Monitoring configuration
- Logging preferences
- Feature toggles
- Security settings

Documentation

`README.md`

- Comprehensive documentation
- Feature overview
- Installation instructions
- Usage examples
- API reference
- Troubleshooting guide
- Architecture documentation

`QUICK_START.md`

- Quick start guide
- 5-minute setup
- Common commands
- Basic troubleshooting
- Next steps

`PROJECT_STRUCTURE.md`

- This file
- Complete file listing
- File descriptions
- Component overview

Examples

`examples/basic_usage.py`

- Example scripts demonstrating API usage
- Device management examples
- Health monitoring examples
- File sharing examples
- Battery monitoring examples

Package Files

`requirements.txt`

- All Python dependencies
- Version specifications
- Organized by category

`setup.py`

- Python package setup script
- Entry point configuration
- Package metadata
- Dependency management

`LICENSE`

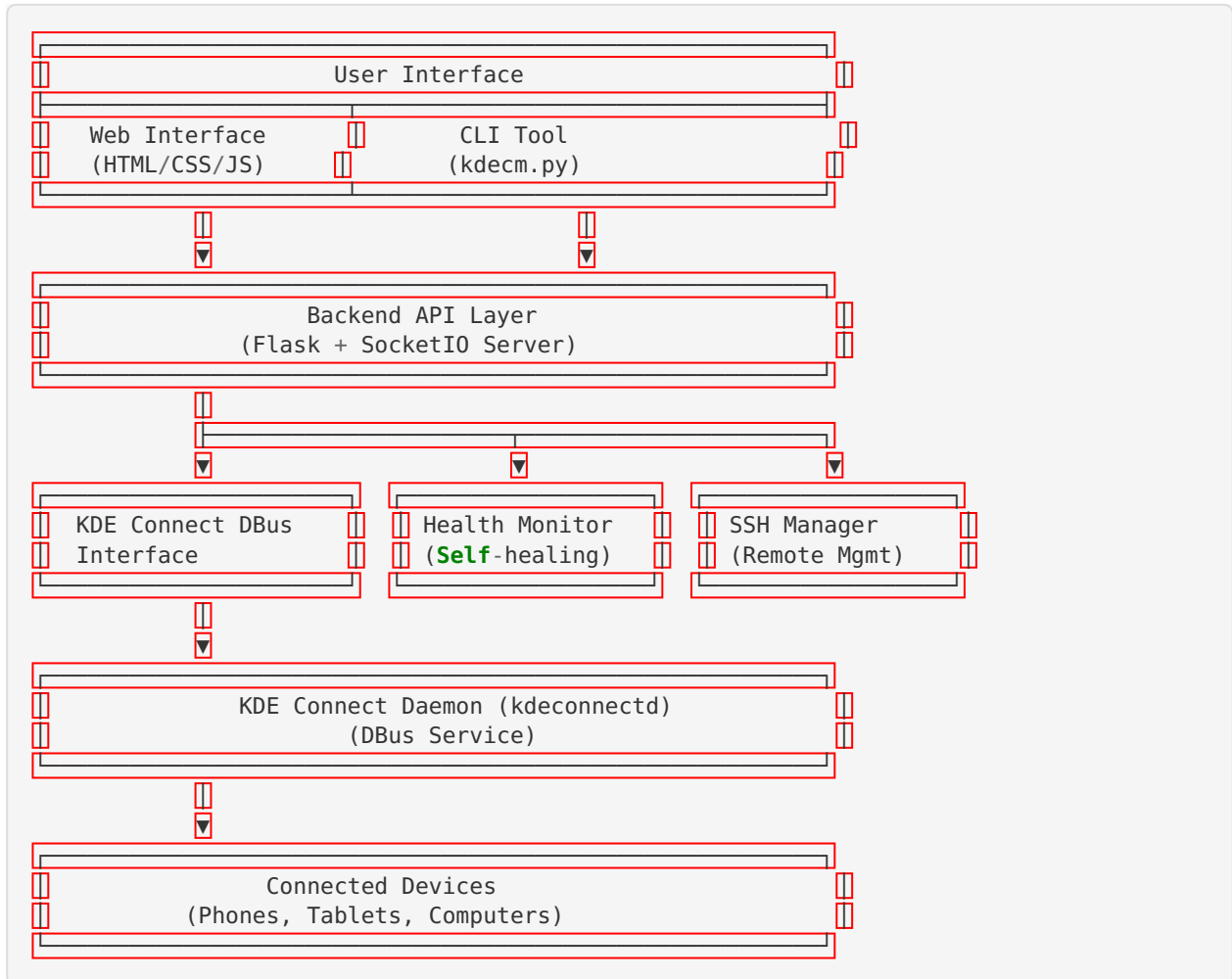
- MIT License
- Copyright information
- Usage terms

.gitignore

- Git ignore patterns
- Python artifacts
- Logs and temporary files
- Configuration files
- IDE files

Architecture Overview

Component Interaction



Data Flow

1. **User Interaction** → Web UI or CLI
2. **API Layer** → Processes requests
3. **DBus Interface** → Communicates with KDE Connect daemon
4. **KDE Connect Daemon** → Manages device connections
5. **Health Monitor** → Continuously monitors system
6. **WebSocket** → Pushes real-time updates to web clients

Key Technologies

- **Python 3.9+**: Backend language
- **Flask**: Web framework

- **SocketIO**: WebSocket support
- **DBus**: IPC with KDE Connect
- **Click**: CLI framework
- **Paramiko**: SSH client
- **psutil**: System monitoring
- **JavaScript**: Frontend logic
- **HTML/CSS**: Web interface
- **systemd**: Service management



Statistics

- **Total Files**: ~30
- **Lines of Code**: ~8,000+
- **Backend Code**: ~3,500 lines
- **Frontend Code**: ~1,500 lines
- **CLI Code**: ~600 lines
- **Documentation**: ~2,500 lines
- **Configuration**: ~400 lines



Development Workflow

1. **Clone repository**
2. **Install dependencies**: `./scripts/install.sh`
3. **Make changes** to code
4. **Test locally**: `kdecn --help`
5. **Start web server**: `kdecn server start`
6. **Run examples**: `python3 examples/basic_usage.py`
7. **Check health**: `kdecn health check`



Deployment

Local Installation

```
./scripts/install.sh
```

Systemd Services

```
systemctl --user enable kdeconnect-manager.service
systemctl --user start kdeconnect-manager.service
```

Remote Deployment

```
from backend.ssh.remote_manager import RemoteKDEConnectManager
# Deploy to remote hosts
```




Notes

- All scripts are executable (`chmod +x`)
 - Logs are stored in `logs/` directory
 - Configuration example in `config/`
 - Service files in `systemd/`
 - Examples in `examples/`
 - Full documentation in `README.md`
-

Project Version: 1.0.0

Last Updated: 2025-10-03

License: MIT